PROJECT REPORT

AI-Powered Customer Intelligence and Retention Platform

Anonymised client: CustomerTech India

Project ID: BCA-S6-MAJOR-AI-001

Submitted in partial fulfilment of the requirements for the degree of

Bachelor of Computer Applications (BCA)

Semester 6 - Major Project

Submitted to

Chandigarh University
Institute of Distance and Online Learning

Submitted by	Puspendu
Enrolment / Student ID	O24BCA160184
Programme	BCA (Online / Distance)
Project guide	_____
Month and year	October 2026

Student Review Note (remove this page before submission)

This report was generated with AI assistance from the supplied dataset, and every number in it comes from code in the project repository. The submission guidelines require authentic, self-owned work, so please read the whole report, rewrite the narrative in your own words, and re-run the code before you submit. Items you must complete yourself:

- Enter the guide's real name, qualification, address, mobile and email in the certificate (Annexure A) and Annexure 6; obtain the guide's and your own signatures.
- Write the Acknowledgement in your own words and name the people who actually helped you.
- Take screenshots of the running app (dashboard and prediction screens) and paste them in Chapter 9 next to the wireframe figure; the wireframes are design sketches, not screenshots.
- Re-run `pytest` and the training scripts on your machine so the Actual Result column reflects your own execution.
- Check whether the university requires its own cover-page template, and confirm the final page count and soft-copy upload link.
- The model is weak (see Chapter 8). Do not change the numbers to look better; the guide asks for honest limitations.
- The Dockerfile has not been built in this environment (no Docker daemon was available); build and run it yourself before claiming deployment evidence.

Acknowledgement

I express my sincere gratitude to my project guide, _____, for the guidance, advice and encouragement provided during this project. I thank the faculty of the Institute of Distance and Online Learning, Chandigarh University, for the curriculum in Business Analytics, Software Engineering, Machine Learning and Cloud-Native Development that made this work possible.

I also thank the creators of the open-source tools used here (Python, pandas, scikit-learn, Streamlit, Plotly, Matplotlib, ReportLab and Docker), and my family and friends for their support. The dataset used in this project was supplied by the university for academic use and is anonymised.

Puspendu
Enrolment No. O24BCA160184

Certificate from Guide (Annexure A)

This is to certify that this project entitled **"AI-Powered Customer Intelligence and Retention Platform"** submitted in partial fulfilment of the degree of **Bachelor of Computer Applications (BCA)** to **Chandigarh University** was done by Mr./Ms. **Puspendu**, Roll No. **O24BCA160184**, and is an authentic work carried out by him/her under my guidance. The matter embodied in this project work has not been submitted earlier for the award of any degree or diploma, to the best of my knowledge and belief.

_____ Signature of the Student	_____ Signature of the Guide
Date: _____	Guide name: _____

Guide name, signature and date must be completed by the actual guide; they are intentionally left blank.

Contents

Acknowledgement 3

Certificate from Guide (Annexure A) 4

Synopsis of the Project 7

1. Objective and Scope of the Project 8

1.1 Business outcome and objectives . . . 8

1.2 Key performance indicators . . . 8

1.3 Measurable target statement . . . 8

1.4 Scope and out-of-scope . . . 8

2. Theoretical Background 9

2.1 Customer churn . . . 9

2.2 Business analytics . . . 9

2.3 Machine-learning concepts used . . . 9

2.4 Evaluation metrics . . . 9

2.5 Software engineering and cloud-native concepts . . . 9

3. Definition of the Problem 10

4. System Analysis and Design versus User Requirements 11

4.1 Requirements . . . 11

4.2 Use cases / user stories . . . 11

4.3 Dataset understanding and data quality . . . 12

4.4 Candidate predictors (no causal claim) . . . 13

4.5 Feature preparation and leakage check . . . 13

5. Exploratory Business Analytics 14

6. Machine Learning Model Development and Evaluation 18

6.1 Algorithm selection . . . 18

6.2 Train/test split . . . 18

6.3 Test-set results . . . 18

6.4 Which error is more costly? . . . 19

6.5 Risk bands . . . 19

6.6 Interpretation of coefficients . . . 20

6.7 Model limitations . . . 20

7. System Design, Planning and Process Logic 21

7.1 Architecture . . . 21

7.2 Data flow diagrams . . . 21

7.3 Entity-relationship diagram . . . 22

7.4 System planning (PERT chart) . . . 22

7.5 Process logic of each module . . . 23
8. Methodology, Implementation and Cloud-Native Design 24
8.1 Methodology . . . 24
8.2 Hardware and software . . . 24
8.3 Cloud-native design and packaging . . . 24
8.4 Maintenance and evaluation . . . 25
8.5 Cost and benefit analysis . . . 25
9. Input and Output Screen Design 26
9.1 Process flow for the user . . . 26
9.2 Security aspects, access rights, back-up and controls (operational manual summary) . . . 26
10. Testing and Validation 28
11. Findings, Recommendations and Conclusion 29
11.1 Evidence-based findings . . . 29
11.2 Recommended retention actions . . . 29
11.3 Future technical enhancements . . . 29
11.4 Conclusion . . . 29
Annexure 1: Background of the Organisation 30
Annexure 2: Data Dictionary 30
Annexure 3: List of Abbreviations, Figures and Tables 30
Annexure 4: References 32
Annexure 5: Soft Copy of the Project 32
Annexure 6: Guide Details 32
Annexure 7: Certificate from Guide 32
Annexure 8: Source Code Printout 33

Synopsis of the Project

Title of the project: AI-Powered Customer Intelligence and Retention Platform

Statement of the problem. CustomerTech India (anonymised) holds growing customer data but has no integrated mechanism to identify which customers are likely to leave, to explain their behaviour, or to turn the insight into timely retention action. Without it, retention effort is reactive and untargeted.

Why this topic was chosen. Customer churn is a high-value, well-defined business problem that combines four subjects of the programme: Business Analytics (KPIs, segmentation), Machine Learning (classification), Software Engineering (requirements, architecture, testing) and Cloud-Native Development (containerised packaging). It lets one project exercise the full path from raw data to a usable, deployable tool.

Objective and scope. The objective is to build a working academic prototype that (1) displays customer KPIs, (2) accepts customer attributes, (3) returns a churn-risk probability, risk band and plain-language recommended action, and (4) is packaged for cloud-ready deployment. The scope covers one anonymised static dataset of 2,200 customers and 15 fields, a single scikit-learn model, a Streamlit interface and a Docker image. Live billing integration, streaming data, authentication and production monitoring are out of scope.

Methodology. The work followed twelve activities: problem definition, requirements, data profiling, exploratory analysis, feature preparation, model development, evaluation, architecture, prototype development, cloud-native packaging, testing and recommendations. The overall churn rate was found to be 29.2%. Three classifiers were compared by five-fold stratified cross-validation on the training set; Logistic Regression was selected for interpretability and highest recall.

Hardware and software. Any laptop with 4 GB RAM or more; Python 3.12, pandas, scikit-learn, Streamlit, Plotly, Matplotlib, openpyxl, joblib, pytest, Git and Docker.

Testing technologies. pytest for twelve automated test cases (data loading, missing values, encoding, split, training, prediction, banding, filter logic, edge cases, input validation, PII exclusion, data quality) and Streamlit's AppTest for a smoke test of the interface.

Contribution and conclusion. The project delivers a reproducible, documented and tested prototype and an honest evaluation. On the held-out test set the model reached accuracy 55.7%, precision 33.5%, recall 51.9%, F1 40.7% and ROC-AUC 0.558. These show only a weak signal in the supplied data, so predictions must be used as decision support, not as guaranteed outcomes. The most useful analytical finding is that churn rises with payment failures, support tickets and low satisfaction, which points to practical retention actions.

1. Objective and Scope of the Project

1.1 Business outcome and objectives

The outcome to improve is **customer retention**, measured as a lower churn rate. The specific objectives are:

- Establish a baseline churn rate and show where churn is concentrated (segment, region, age group and behavioural drivers).
- Build and honestly evaluate a classification model that estimates each customer's churn probability.
- Deliver a prototype that links analysis to action: KPI dashboard, risk prediction and recommended retention action.
- Design the application so that it can be containerised and scaled in a cloud environment.
- Produce full evidence: requirements, tests, evaluation output, architecture and source code.

1.2 Key performance indicators

KPI	Definition	Baseline (this dataset)
Overall churn rate	Churned customers / total customers x 100	29.2% (643 of 2200)
Churn rate by segment / region / age	Same formula within each group	Chapter 6
Avg satisfaction of churned customers	Mean Satisfaction_Score where Churned = Yes	2.82 (vs 3.12 for retained)
Model recall on churners	TP / (TP + FN)	51.9%
High-risk band size	Customers scored 0.55 or above	131 of 440 test customers

Table 1: KPIs used in the project

1.3 Measurable target statement

Target: reduce the churn rate from the baseline of 29.2% by targeting retention actions at customers in the High risk band. A reduction of even one percentage point corresponds to about 22 customers retained out of 2,200. The prototype cannot prove the reduction because no live intervention is run; it provides the baseline, the targeting list and the KPI definitions against which a future campaign can be measured.

1.4 Scope and out-of-scope

In scope: analysis of the supplied anonymised dataset; one churn model; Streamlit dashboard and prediction form; local SQLite/CSV data layer; Dockerfile; requirements, tests and documentation.

Out of scope (deliberately):

- Integration with live billing, CRM or messaging systems.
- Real-time or streaming data; the dataset is a static snapshot.
- User authentication, role-based access and production-grade security hardening.
- Measuring the real-world effect of retention campaigns (no A/B test was run).
- Cost-sensitive optimisation using real margin or campaign-cost data (none was supplied).
- Causal claims: the analysis reports association, not causation.

2. Theoretical Background

2.1 Customer churn

Churn is the loss of a customer. Retaining an existing customer is generally cheaper than acquiring a new one, so businesses track churn rate and try to predict it in advance. Churn is modelled as a supervised **binary classification** problem: the label is Churned (Yes/No) and the inputs are demographic, transactional, support and engagement attributes.

2.2 Business analytics

Descriptive analytics summarises what has happened (churn rate by group), diagnostic analytics asks where and why, and predictive analytics estimates what is likely next. This project uses all three in sequence. **Churn rate = churned customers / total customers x 100.**

2.3 Machine-learning concepts used

- **Logistic Regression** models the log-odds of churn as a linear combination of features: $p = 1 / (1 + e^{-(b0 + b1x1 + ... + bkxk)})$. Coefficients are directly interpretable, which suits a business audience (Hosmer and Lemeshow, 2013).
- **Random Forest** averages many decision trees trained on resampled data and features, capturing non-linear effects at the cost of interpretability (Breiman, 2001). **Gradient Boosting** builds trees sequentially to correct earlier errors.
- **One-hot encoding** converts a category into binary columns so that no false order is implied; **standardisation** rescales numeric fields to zero mean and unit variance so coefficients are comparable.
- **Stratified train/test split** keeps the class proportion equal in both parts. **Cross-validation** estimates performance by rotating validation folds inside the training set.
- **Class weighting** penalises errors on the minority class (churners) more, improving recall at some cost to precision.
- **Target leakage** occurs when a feature contains information only available after the outcome; it inflates results and must be checked.

2.4 Evaluation metrics

From the confusion matrix (TP, FP, TN, FN): **Accuracy** = (TP+TN)/all; **Precision** = TP/(TP+FP); **Recall** = TP/(TP+FN); **F1** = 2PR/(P+R); **ROC-AUC** is the probability that a random churner is scored above a random non-churner (0.5 = no skill, 1.0 = perfect).

2.5 Software engineering and cloud-native concepts

Requirements are written as testable statements with acceptance criteria. Layered architecture separates the user interface, business logic and data so each can change independently. Cloud-native design favours stateless containers, externalised configuration (environment variables), horizontal scaling behind a load balancer, and centralised logging and monitoring. Docker packages the application and its dependencies into a portable image.

3. Definition of the Problem

CustomerTech India has customer, transaction, support and engagement data but cannot (a) quantify churn by group, (b) estimate an individual customer's risk, or (c) connect the estimate to a concrete action. The problem is therefore to translate a business question - *"which customers are likely to leave, and what should we do?"* - into a software-and-analytics solution.

Users: a retention or customer-success analyst who needs a dashboard and a quick risk check; a manager who needs KPIs. **Inputs:** the 14 customer attributes. **Outputs:** KPIs, churn probability, risk band (Low/Medium/High) and a recommended action.

Constraints: self-driven academic project of about 600 man-hours; one anonymised static dataset; no cloud credentials, so deployment is local and cloud-ready rather than cloud-hosted; predictions are decision support only.

Assumptions: (i) each row is one distinct customer; (ii) the Churned label is the true outcome observed after the behavioural window; (iii) the data are representative of the client's customer base. None of these can be verified from the files supplied.

4. System Analysis and Design versus User Requirements

4.1 Requirements

The five starter requirements from the supplied template were retained and extended with four of my own. Every requirement has an acceptance criterion and is traced to the module and test that satisfy it.

ID	Requirement	Type	Acceptance criteria	Status / Evidence
FR-01	Dashboard displays core customer KPIs	Functional	Dashboard shows customer count, churn rate, avg satisfaction of churned customers and avg spend; values match pandas calculations (2,200; 29.2%; 2.82; 4,815).	Met - app.py tab 1; AppTest smoke test (TC-13)
FR-02	System accepts customer attributes for churn prediction	Functional	Form accepts all 13 model inputs; out-of-range or missing values are rejected with a clear message.	Met - app.py tab 2; TC-10
FR-03	System returns churn-risk output	Functional	Returns probability between 0 and 1 and a band Low/Medium/High for any valid input.	Met - predict.py; TC-06, TC-07, TC-09
FR-04	Dashboard can be filtered by segment and region	Functional	Selecting segment/region updates all KPIs and charts; empty selection shows a warning, not an error.	Met - TC-08, TC-13
FR-05	System shows a recommended action for each risk band	Functional	Each of Low/Medium/High maps to a distinct plain-language action.	Met - predict.ACTIONS; TC-07
FR-06	System displays top factors raising a selected customer's risk	Functional	Top three contributing features (coefficient x standardised value) are listed.	Met - predict.top_drivers
NFR-01	Application should respond reliably under normal academic demo load	Non-functional	Single-user demo: prediction returns in under 1 second; app runs without exceptions on load.	Met for single user - not load-tested
NFR-02	Solution should avoid exposing personally identifiable information	Non-functional	Customer_ID is excluded from model inputs and from the prediction form; dataset contains only anonymised IDs.	Met - TC-11
NFR-03	Configuration must be externalised	Non-functional	Model path is read from the MODEL_PATH environment variable with a safe default; no secrets in code.	Met - predict.py, Dockerfile

Table 2: Requirements specification with acceptance criteria

NFR-01 is satisfied only for a single local user; no load test was run, so scalability is argued from the design (Chapter 7) rather than demonstrated.

4.2 Use cases / user stories

- As a retention analyst, I want to filter the dashboard by segment and region so that I can see where churn is highest.
- As an analyst, I want to enter a customer's attributes and get a risk band so that I can decide whether to contact them.
- As a manager, I want a recommended action for each risk band so that the team acts consistently.
- As a data owner, I want personal identifiers kept out of the tool so that privacy is protected.

4.3 Dataset understanding and data quality

The dataset (Customer Data sheet) has **2,200 rows and 15 columns**. Customer_ID is unique (no duplicate IDs or rows). There are **no missing values**, no negative numbers, Discount_Usage_Pct lies within 0-80 (never outside 0-100) and Satisfaction_Score lies within 1-5. The target is imbalanced but not extreme: 1,557 retained (70.8%) and 643 churned (29.2%).

Field	Type	Observed range / values	Mean
Tenure_Months	Integer	1 to 60	30.89
Monthly_Spend_INR	Numeric	605 to 8996	4815.42
Transactions_90D	Integer	0 to 18	8.96
Support_Tickets_90D	Integer	0 to 7	3.48
Avg_Response_Hours	Numeric	1 to 30	15.48
Satisfaction_Score	Integer	1 to 5	3.04
App_Logins_30D	Integer	0 to 35	17.65
Days_Since_Last_Purchase	Integer	1 to 120	59.77
Discount_Usage_Pct	Percentage	0 to 80	40.11
Payment_Failures_90D	Integer	0 to 4	1.97
Age_Group	Category	18-24, 25-34, 35-44, 45-54, 55+	-
Region	Category	Central, East, North, South, West	-
Customer_Segment	Category	Growth, Premium, Value	-

Table 3: Profile of the dataset (observed values)

Observation on the supplied schema. The build prompt described four age bands and four regions, but the actual file has **five age groups** (18-24, 25-34, 35-44, 45-54, 55+) and **five regions** (adding South). The project follows the actual data.

Data-quality risks and limitations.

- Class imbalance (29.2% churn): accuracy alone is misleading because predicting "No" for everyone scores 70.7% on the test set.
- Several numeric fields look uniformly distributed (for example Avg_Response_Hours 1-30, Discount_Usage_Pct 0-80), which suggests the data may be synthetic or heavily anonymised; real-world patterns may be weaker or different.
- Support_Tickets_90D has a narrow range (0-7) and Payment_Failures_90D (0-4), so these behave almost like categories.
- The dataset is a single snapshot; time order is unknown, so a time-based split and drift checks were not possible.
- Whether the behavioural fields were measured before the churn event is assumed, not verified (leakage risk discussed in 4.5).

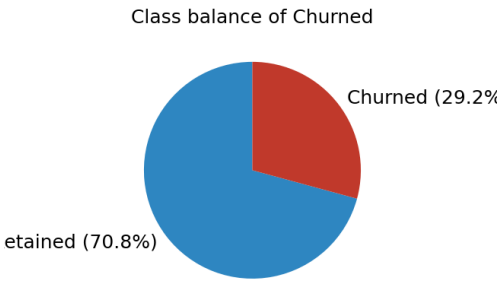


Figure 1: Class balance of the target variable

4.4 Candidate predictors (no causal claim)

- **Payment_Failures_90D, Support_Tickets_90D, Avg_Response_Hours:** friction with billing and service may be associated with leaving.
- **Satisfaction_Score:** a direct sentiment measure, expected to be associated with retention.
- **Days_Since_Last_Purchase, Transactions_90D, App_Logins_30D:** recency and engagement; falling activity often precedes churn.
- **Tenure_Months, Monthly_Spend_INR, Discount_Usage_Pct:** relationship length, value and price sensitivity.
- **Age_Group, Region, Customer_Segment:** context variables used to compare groups and as model inputs.

4.5 Feature preparation and leakage check

Step	Decision	Justification
Identifier	Customer_ID excluded	Unique identifier, not predictive, and a privacy risk (NFR-02).
Categorical fields	One-hot encoding of Age_Group, Region, Customer_Segment (handle_unknown=ignore)	Regions and segments have no natural order; avoids implying one. Unknown categories at prediction time do not crash the model.
Numeric fields	StandardScaler on all 10 numeric inputs	Logistic Regression is scale-sensitive and standardised coefficients are comparable.
Missing values	Median imputation implemented, 0 values needed	None are present, but the pipeline remains robust.
Target	Churned mapped Yes=1, No=0	Binary classification target.
Final feature set	13 raw fields -> 23 model columns (3 categories become 13 dummies + 10 numeric)	No other fields were dropped.

Table 4: Feature preparation decisions

Target-leakage check. Each feature describes behaviour in a window (90 days or 30 days) or a status (tenure, latest satisfaction, recency). None is by definition a post-churn event such as a cancellation code or final-bill field. However, Days_Since_Last_Purchase and Satisfaction_Score could in principle be recorded *after* a customer has already disengaged. This cannot be verified from the data dictionary, so it is listed as a limitation rather than assumed away.

5. Exploratory Business Analytics

Overall churn rate = 643 / 2,200 x 100 = 29.2%. Differences between segments, regions and age groups are small; the strongest patterns appear in behavioural fields.

Customer segment	Customers	Churned	Churn rate
Growth	755	227	30.1%
Premium	729	216	29.6%
Value	716	200	27.9%

Table 5: Churn rate by customer segment

Region	Customers	Churned	Churn rate
Central	418	112	26.8%
East	439	142	32.3%
North	412	112	27.2%
South	477	140	29.4%
West	454	137	30.2%

Table 6: Churn rate by region

Age group	Customers	Churned	Churn rate
18-24	439	138	31.4%
25-34	411	117	28.5%
35-44	452	134	29.6%
45-54	449	127	28.3%
55+	449	127	28.3%

Table 7: Churn rate by age group

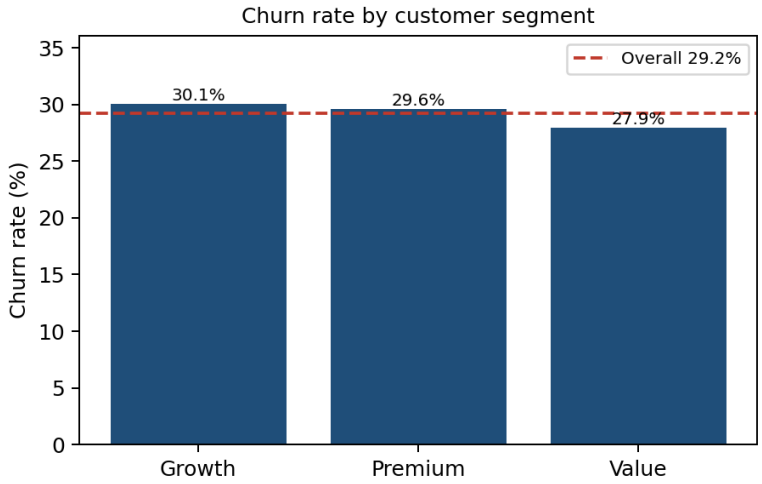


Figure 2: Churn rate by customer segment

Business interpretation. Growth customers churn slightly more (30.1%) than Premium (29.6%) and Value (27.9%), but the spread is about two percentage points. Segment alone is not a strong lever; it should not be the main targeting rule.

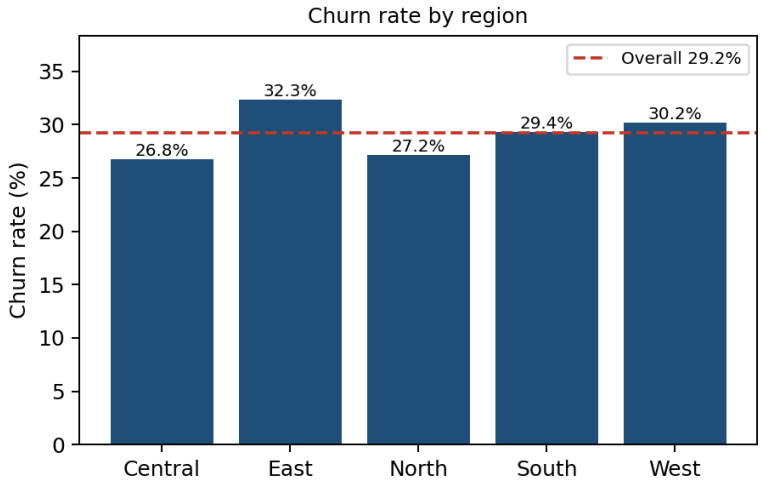


Figure 3: Churn rate by region

Business interpretation. East has the highest churn (32.3%) and Central the lowest (26.8%), a gap of about five points. East is a candidate for a regional review of service and delivery, though with roughly 440 customers per region this gap could partly be chance.

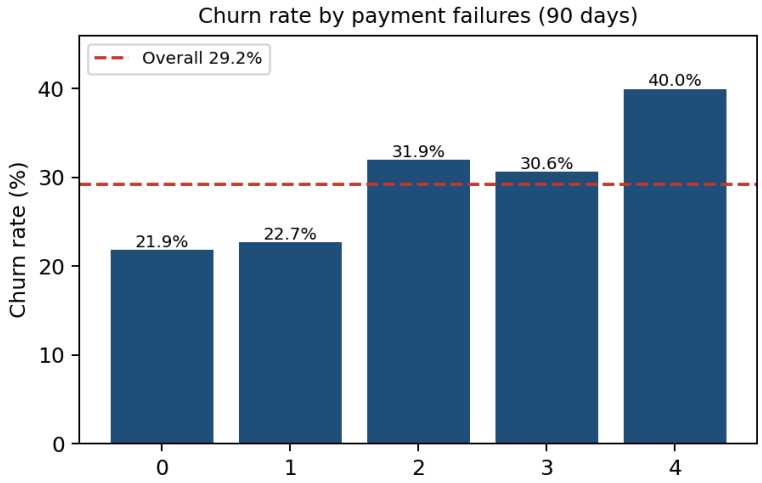


Figure 4: Churn rate by payment failures in the previous 90 days

Business interpretation. This is the clearest pattern: churn is 21.9% with no payment failure and 40.0% with four failures, almost double. Payment friction (failed cards, retries, billing errors) is strongly associated with leaving and is actionable.

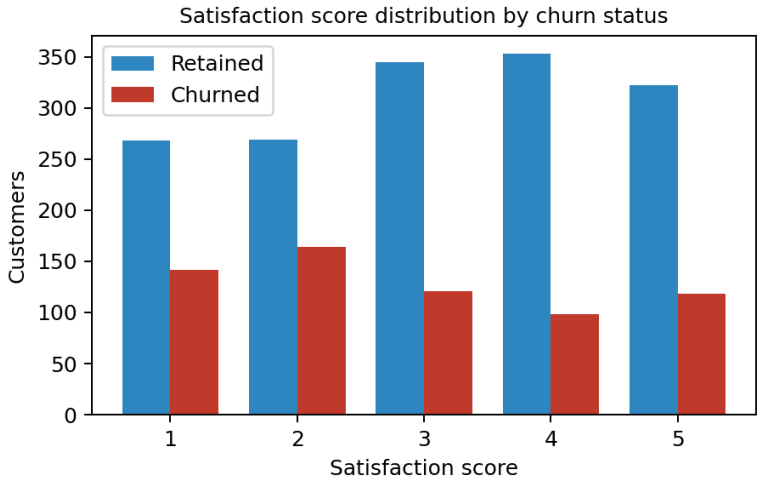


Figure 5: Satisfaction score distribution split by churn status

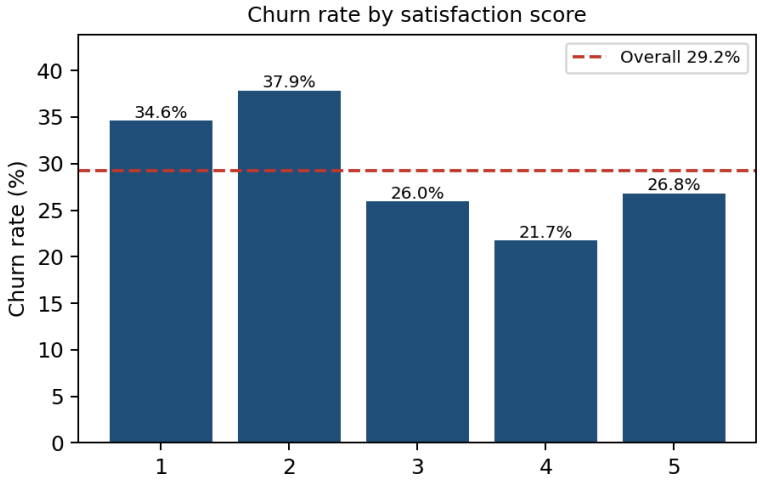


Figure 6: Churn rate by satisfaction score

Business interpretation. Churn is highest at scores 1 and 2 (34.6% and 37.9%) and lowest at 4 (21.7%). The pattern is not perfectly smooth: score 5 shows 26.8%, higher than score 4, so satisfaction is informative but not a clean one-way predictor.

Support tickets. Churn is 21.6-25.4% for customers with 0-3 tickets but 33.3-37.0% for those with 4 or more, which suggests a threshold effect around four tickets in 90 days.

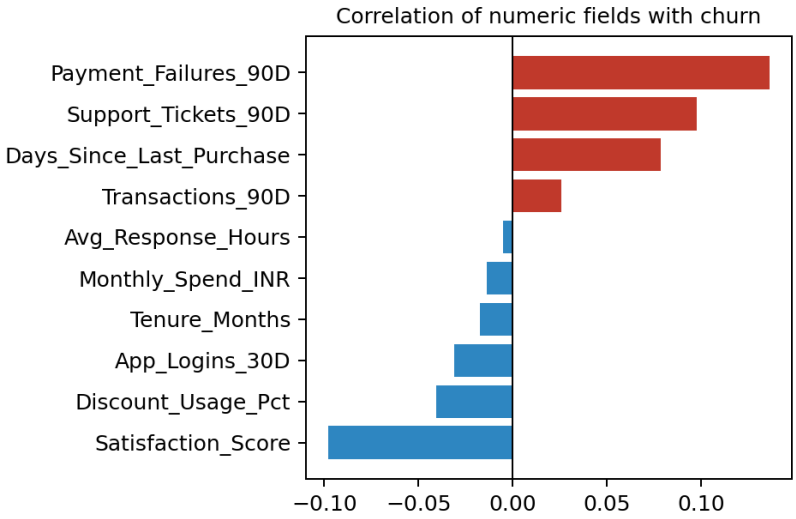


Figure 7: Correlation of numeric fields with churn

Interpretation. All correlations are small in absolute terms. The largest are Payment_Failures_90D (+0.14), Support_Tickets_90D (+0.10), Satisfaction_Score (-0.10) and Days_Since_Last_Purchase (+0.08). Tenure, spend and response time are close to zero. This foreshadows a modest ceiling on model accuracy.

6. Machine Learning Model Development and Evaluation

6.1 Algorithm selection

Three classifiers were compared using five-fold stratified cross-validation on the 1,760-row training set only. The test set was not used for selection.

Model	ROC-AUC	Recall	Precision	F1	Accuracy
Logistic Regression	0.625	0.588	0.379	0.461	0.598
Random Forest	0.625	0.438	0.411	0.423	0.652
Gradient Boosting	0.613	0.080	0.425	0.134	0.699

Table 8: Cross-validation results on the training set (mean of 5 folds)

Choice: Logistic Regression with balanced class weights. It ties Random Forest on ROC-AUC (0.625), has the highest recall (0.588) - important because missing a churner is costly - and is interpretable, which matters because the output must justify an action to a business user. The dataset is small (2,200 rows) with mostly numeric inputs and weak signals, so a more complex model offers little benefit. Gradient Boosting at the default 0.5 threshold almost never predicted churn (recall 0.08), so it is unsuitable without threshold tuning.

6.2 Train/test split

An 80/20 **stratified** split with random_state 42 gave 1,760 training and 440 test rows. Stratification keeps the churn rate at 29.2% in training and 29.3% in test, which matters because the minority class is only 29%. 80/20 keeps enough training data for 23 features while leaving 129 churners in the test set for evaluation. Model parameters: LogisticRegression(max_iter=1000, class_weight='balanced'), preceded by one-hot encoding and standardisation in a single scikit-learn Pipeline saved to models/churn_model.pkl.

6.3 Test-set results

Metric	Value	Business interpretation
Accuracy	55.7%	245 of 440 customers classified correctly. This is **below** the 70.7% obtained by predicting 'stays' for everyone, because the model deliberately flags more customers to catch churners.
Precision	33.5%	Of 200 customers flagged as likely to churn, only 67 actually did, so about two in three retention contacts go to customers who would have stayed.
Recall	51.9%	The model finds 67 of 129 real churners (about half); 62 churners are missed.
F1-score	40.7%	Balance of precision and recall; moderate-to-low.
ROC-AUC	0.558	Only slightly above 0.5 (random). A bootstrap 95% interval on the test set is about 0.50-0.62, so the true ability is uncertain.

Table 9: Model evaluation on the held-out test set

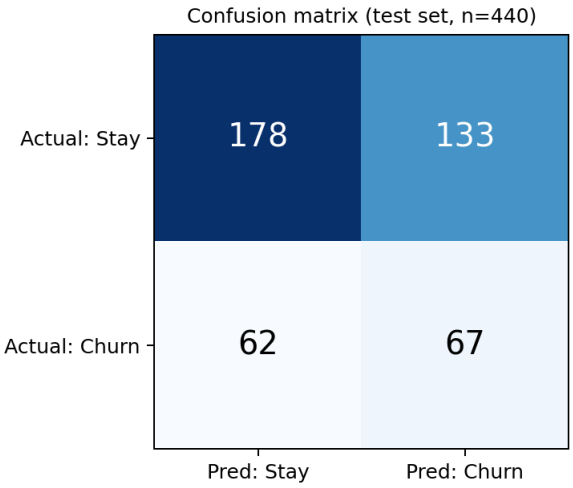


Figure 8: Confusion matrix on the test set: TN=178, FP=133, FN=62, TP=67

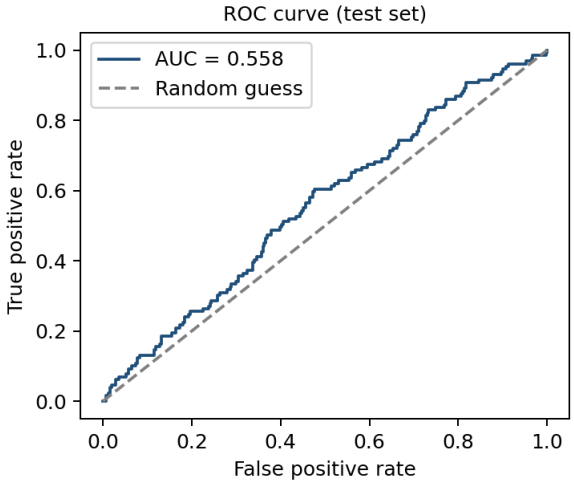


Figure 9: ROC curve on the test set

6.4 Which error is more costly?

A false negative is usually more costly. A false negative is a customer who really churns but is not flagged, so the company loses that customer's future revenue (average spend is about INR 4,815 per month) and gets no chance to intervene. A false positive costs only a retention contact or small incentive offered to a customer who would have stayed. Unless the incentive is very expensive, recall deserves priority over precision, which is why class weights were used. The balance should be revisited with real campaign-cost data.

6.5 Risk bands

Probabilities are grouped into Low (below 0.35), Medium (0.35 to below 0.55) and High (0.55 or above). The thresholds are my design choice, not derived from cost data.

Band	Test customers	Actual churn rate
Low	77	22.1%
Medium	232	30.2%
High	131	32.1%

Table 10: Actual churn rate by predicted risk band (test set)

The ordering is correct (22.1% to 30.2% to 32.1%), but the separation between Medium and High is small. The model gives a modest ranking signal, not a precise one.

6.6 Interpretation of coefficients

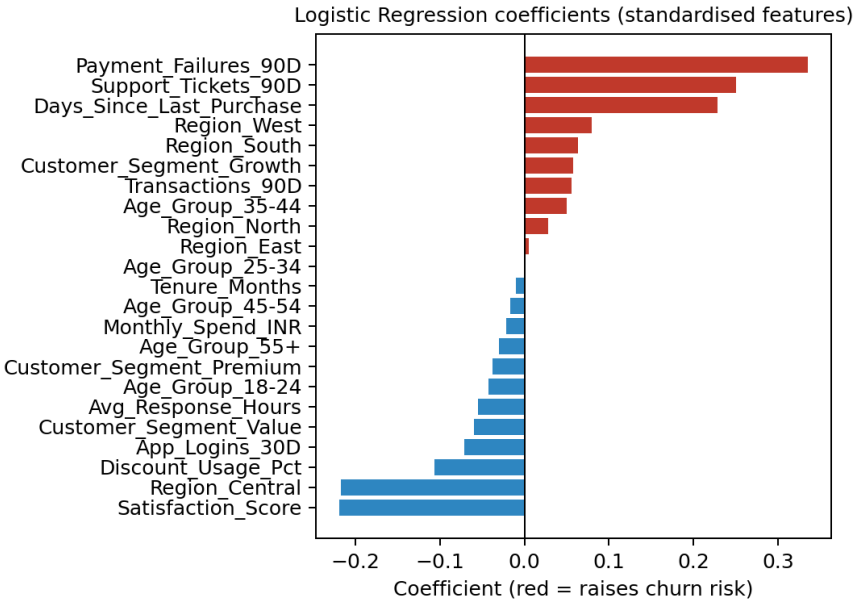


Figure 10: Logistic Regression coefficients on standardised features

The strongest risk-raising features are Payment_Failures_90D (+0.34), Support_Tickets_90D (+0.25) and Days_Since_Last_Purchase (+0.23); the strongest protective features are Satisfaction_Score (-0.22) and Region_Central (-0.22). This agrees with the exploratory findings and is reassuring, but coefficients show association, not cause.

6.7 Model limitations

- Weak signal: the test ROC-AUC (0.558) is lower than cross-validated AUC (0.625); with 440 test rows the difference is within sampling noise but the performance is clearly modest.
- Accuracy is below the naive baseline; the model should be judged on recall and ranking, not accuracy.
- No hyper-parameter tuning or threshold optimisation was done; a small gain may be possible but the data appear to contain limited predictive signal.
- Single static snapshot; performance on future data is unknown and drift is not monitored.
- Predictions are decision support and must be combined with human judgement.

7. System Design, Planning and Process Logic

7.1 Architecture

The system uses a layered design: a Streamlit UI layer, a business-logic and model-serving layer, a data layer and an offline training pipeline.

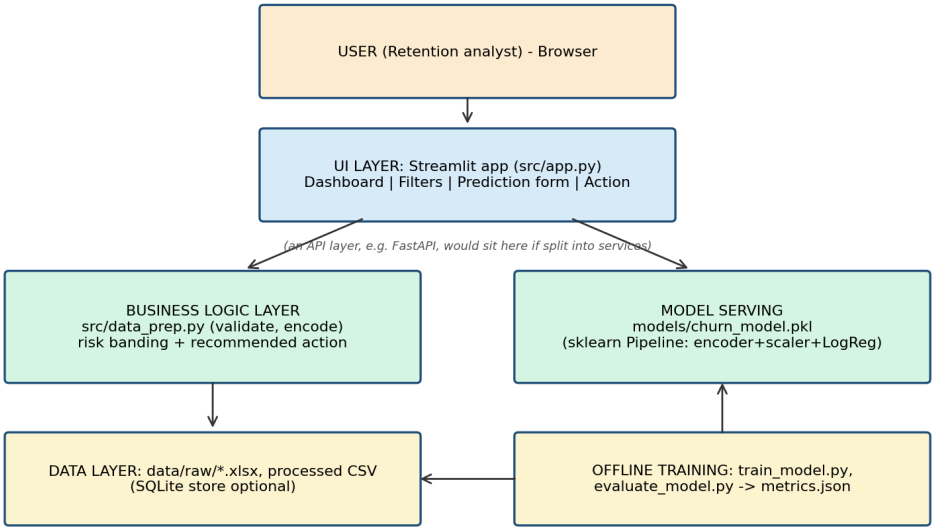


Figure 11: System architecture

Module boundaries. data_prep.py (data layer) has no dependency on predict.py or app.py; predict.py (business logic) does not import the UI; app.py is the only module that imports Streamlit. If the system were split into services, a REST API (for example FastAPI exposing POST /predict) would sit between app.py and predict.py.

7.2 Data flow diagrams

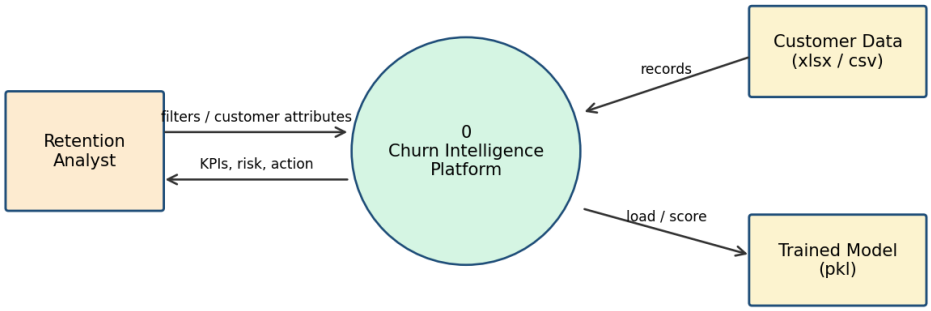


Figure 12: Context-level (Level 0) data flow diagram

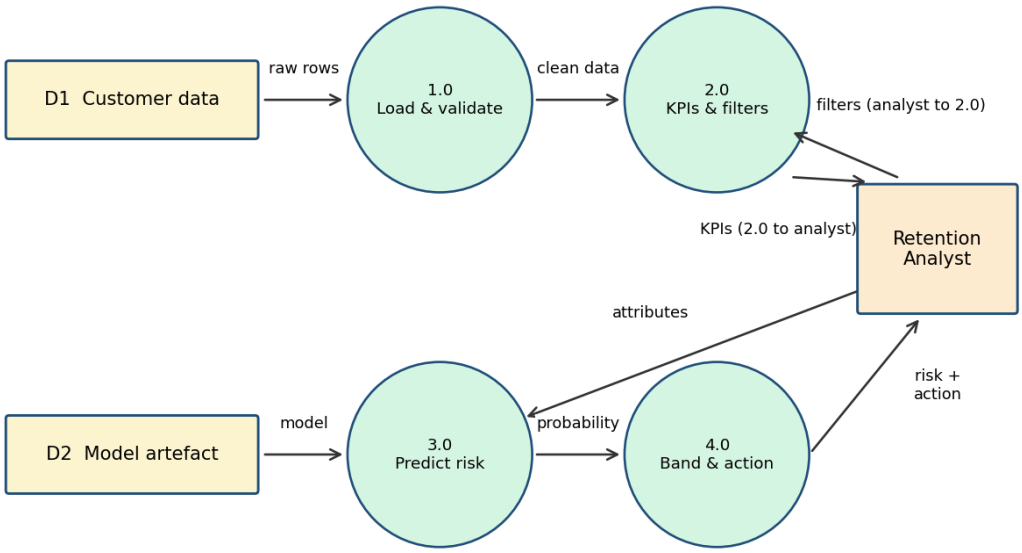
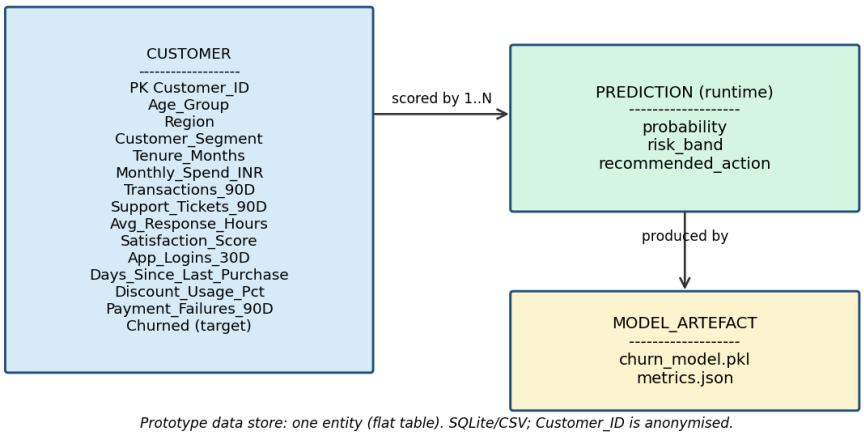


Figure 13: Level 1 data flow diagram

7.3 Entity-relationship diagram



Prototype data store: one entity (flat table). SQLite/CSV; Customer_ID is anonymised.

Figure 14: Entity-relationship diagram

7.4 System planning (PERT chart)

The project of roughly 600 man-hours was planned as the activities below. Durations are planning estimates in working days, not measured values.

Activity	Description	Predecessor	Est. days
A	Problem definition and requirements	-	4
B	Data profiling and exploratory analysis	A	6
C	Feature preparation and model development	B	6
D	Model evaluation and interpretation	C	3
E	Application prototype build	D	7
F	Docker packaging and cloud design	D	3

Activity	Description	Predecessor	Est. days
G	Testing and validation	E	3
H	Final report and review	F, G	6

Table 11: Activity list for PERT chart

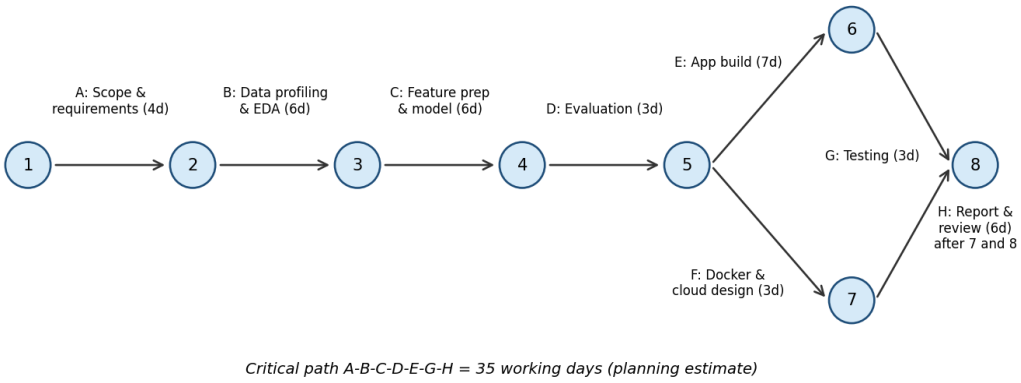


Figure 15: PERT chart (critical path A-B-C-D-E-G-H, 35 days of path duration)

7.5 Process logic of each module

Module	Process logic
data_prep.py	1) Read the Excel file. 2) validate(): count missing values, duplicate IDs, out-of-bound values. 3) clean(): strip text, drop duplicate IDs, median-impute numeric gaps. 4) Provide ColumnTransformer (one-hot + scaler) and stratified split.
train_model.py	1) Clean data and split 80/20 stratified. 2) For each candidate: 5-fold stratified CV on training data. 3) Fit chosen pipeline on full training set. 4) Save to models/churn_model.pkl and CV results to JSON.
evaluate_model.py	1) Load model. 2) Predict probabilities on test set; threshold 0.5. 3) Compute accuracy, precision, recall, F1, AUC, confusion matrix, risk-band table. 4) Save metrics.json and figures.
predict.py	1) validate_input(): all fields present and within bounds, else ValueError. 2) predict_proba. 3) risk_band(): below 0.35 Low, below 0.55 Medium, else High. 4) Return probability, band, action. 5) top_drivers(): coefficient x standardised value, top three.
app.py	Tab 1: load data, apply segment/region filters, show KPIs and charts, warn if no rows. Tab 2: collect inputs in a form; on submit call predict_one and display probability, band, action and top factors; show validation errors.

Table 12: Process logic per module

8. Methodology, Implementation and Cloud-Native Design

8.1 Methodology

An iterative, evidence-driven method was used: define the problem, specify requirements, profile data, explore, prepare features, model, evaluate, design, build, package, test, and recommend. Each step produced a stored artefact (document, figure, script or metric file) and the repository was committed to Git after each stage.

8.2 Hardware and software

Item	Specification
Hardware	Standard laptop/desktop, 4 GB RAM or more, no GPU required.
Operating system	Windows, macOS or Linux (development used Ubuntu 24).
Language and libraries	Python 3.12; pandas, numpy, scikit-learn, matplotlib, plotly, openpyxl, joblib.
Application framework	Streamlit.
Data store	Excel/CSV files; SQLite is the planned persistent store.
Testing	pytest; Streamlit AppTest.
Version control / packaging	Git; Docker.

Table 13: Hardware and software

8.3 Cloud-native design and packaging

The provided Dockerfile builds a slim Python 3.12 image, installs dependencies, copies the source, model artefact and processed data, runs as a non-root user, exposes port 8501 and defines a health check. **This Dockerfile has not been built or run in the environment where this report was prepared;** it must be built locally (`docker build -t churn-platform .` then `docker run -p 8501:8501 churn-platform`) before it is cited as deployment evidence.

Components and needs. Application service: stateless Streamlit container. Model artefact: churn_model.pkl, mounted as a read-only volume or pulled from object storage at start-up. Configuration: environment variable MODEL_PATH (and any future settings) rather than hard-coded values. Persistent data: customer data on a mounted volume or managed database, never baked into the container for production.

Scalability. Because the container is stateless, several replicas can run behind a load balancer; scaling is horizontal and driven by CPU or request count. Model inference for one record is fast (well under a second locally), so the first bottleneck would be data access, not the model.

Secrets. Credentials (database passwords, API keys, if later added) would be injected at runtime from the platform's secret manager, never committed to Git (.env is in .gitignore).

Monitoring and logging. Write structured logs to stdout for collection by the platform; track request count, latency and errors; monitor the share of customers in each risk band and the proportion of invalid inputs. Monitor data drift and periodically compare predictions with actual churn to decide when to retrain.

8.4 Maintenance and evaluation

- **Retraining:** retrain when new labelled data is available or when monitored recall falls; run train_model.py then evaluate_model.py.
- **Versioning:** keep each model artefact and metrics.json under a version tag in Git or object storage.
- **Back-up:** back up the data volume and model artefacts on a schedule; the code is protected by Git.
- **Regression testing:** run pytest before each release; add tests for each defect found.
- **Evaluation:** track the KPIs in Table 1 on each refresh and compare with the baseline churn rate.

8.5 Cost and benefit analysis

Costs. All tools are open source, so licence cost is zero. The main cost is student effort of about 600 man-hours (per the submission guidelines) plus ordinary computing and an internet connection. Running costs for a small cloud deployment (one container and storage) would be modest but were not quoted, so no rupee figure is claimed.

Benefits (illustrative, assumption-based). Average monthly spend in the dataset is INR 4,815. If a retention programme guided by the tool saved 1% of customers (22 of 2,200), the revenue retained would be about 22 x 4,815 = **INR 105,900 per month**, before the cost of incentives. This is a worked example of the arithmetic, not a forecast: the effect size, campaign cost and customer margin are unknown. Qualitative benefits include consistent, documented targeting and a reusable analytics pipeline.

9. Input and Output Screen Design

The interface has two tabs. The wireframes below are design sketches; replace or supplement them with real screenshots from the running application.

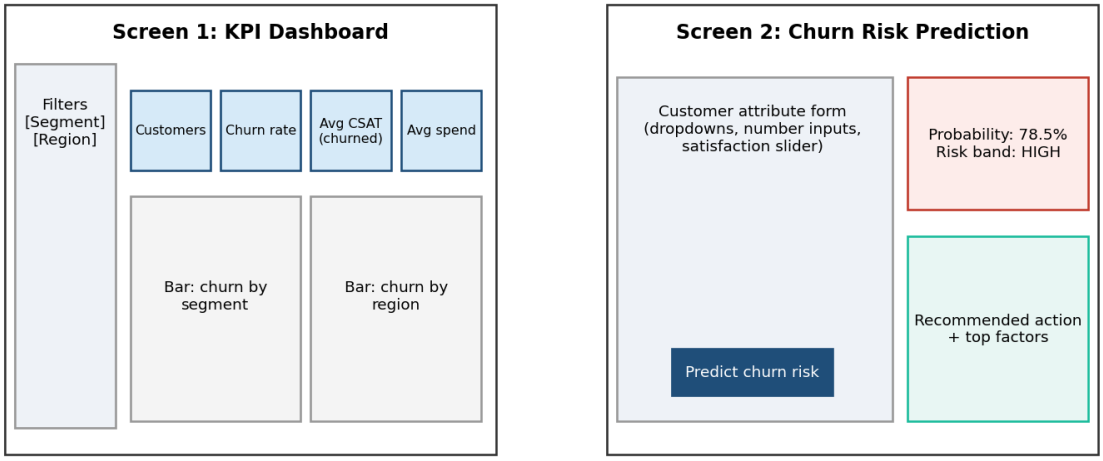


Figure 16: Screen designs: KPI dashboard and churn risk prediction

Inputs: sidebar multi-select for segment and region; prediction form with three dropdowns (age group, region, segment), nine numeric inputs and a satisfaction slider (1-5). **Outputs:** four KPI tiles, three bar charts, churn probability, colour-coded risk band, recommended action and top contributing factors.

Worked examples produced by the prototype (illustrative customers I constructed to show the two extremes, not real records):

Customer profile	Probability	Band	Top factors
18-24, East, Growth, tenure 6, 6 tickets, 4 payment failures, satisfaction 2, 100 days since purchase	78.5%	High	Payment_Failures_90D, Support_Tickets_90D, Days_Since_Last_Purchase
45-54, Central, Value, tenure 48, 1 ticket, 0 payment failures, satisfaction 4, 15 days since purchase	16.9%	Low	(no strong risk factor)

Table 14: Example predictions

9.1 Process flow for the user

The user opens the dashboard, filters by segment or region to see the churn rate, switches to the prediction tab, enters or selects customer attributes, presses Predict, reads the probability and band, and follows the recommended action. Map of features to requirements: KPI tiles - FR-01; filters - FR-04; form - FR-02; probability and band - FR-03; action text - FR-05; top factors - FR-06.

9.2 Security aspects, access rights, back-up and controls (operational manual summary)

- **Install:** create a virtual environment, `pip install -r requirements.txt`, run `python -m src.data_prep`, `python -m src.train_model`, then `streamlit run src/app.py`.
- **Privacy:** only anonymised IDs exist; Customer_ID is not used by the model or form; no data are sent to external services.

- **Access rights:** the prototype has no login. In production, access should be restricted by role (analyst: view and predict; administrator: retrain and deploy) via the organisation's identity provider.
- **Input controls:** numeric bounds and required fields are enforced before prediction.
- **Back-up:** copy models/ and data/ regularly; use Git for code.
- **Troubleshooting:** if the app reports a missing model file, run train_model.py; if the data file is missing, place customer_data.xlsx in data/raw/.

10. Testing and Validation

Methodology. Black-box functional tests of each module and white-box checks of data handling were automated in pytest (file tests/test_platform.py). All twelve cases were executed in the development environment and passed (12 passed in about 4.5 seconds). A thirteenth check used Streamlit's AppTest to load the full app without exceptions and apply a filter.

ID	Module	Test scenario	Expected result	Actual result	Pass/Fail
TC-01	Data loading	Load customer_data.xlsx	2,200 rows and 15 columns; first column Customer_ID	As expected	Pass
TC-02	Cleaning	Insert a missing Monthly_Spend_INR value and run clean()	No missing values remain	No missing values	Pass
TC-03	Encoding	Fit the preprocessor	Customer_ID absent; 23 output columns	23 columns, no ID	Pass
TC-04	Split	Stratified 80/20 split	440 test rows; churn rate within 1 point of train	440 rows; 29.3% vs 29.2%	Pass
TC-05	Training	Fit Logistic Regression pipeline	Trains without error, supports predict_proba	Trained successfully	Pass
TC-06	Prediction	Predict a valid customer	Probability in [0,1] and a valid band	Valid output	Pass
TC-07	Risk logic	Probabilities 0.1, 0.4, 0.9	Low, Medium, High; action text present	Low, Medium, High; action returned	Pass
TC-08	Filtering	Filter Premium + West	Non-empty subset containing only Premium	Subset returned correctly	Pass
TC-09	Edge case	New customer with tenure 0	Valid probability returned	Valid probability	Pass
TC-10	Validation	Satisfaction 9; missing Region	ValueError raised for each	ValueError raised	Pass
TC-11	Privacy	Check feature list	Customer_ID not a model feature	Not present	Pass
TC-12	Data quality	Run validate() on raw data	0 missing, 0 duplicate IDs, 0 out-of-bound discounts	All zero	Pass
TC-13	UI smoke	Load app via AppTest; filter to Premium	No exceptions; KPIs update (2,200 to 729 customers)	No exceptions; 729 customers, 29.6%	Pass

Table 15: Test report

Defects and limitations found. (1) The first run of the app produced deprecation warnings for the Streamlit parameter use_container_width; this was corrected by switching to width='stretch'. (2) Testing showed that top_drivers can list a zero-contribution dummy variable for low-risk customers (for example Age_Group_18-24 at 0.00), which is not informative; it is documented as a limitation of ranking by contribution when no strong driver exists. (3) Load, browser and multi-user tests, and a Docker build test, were not performed.

11. Findings, Recommendations and Conclusion

11.1 Evidence-based findings

No.	Finding	Evidence
1	Payment failures are strongly associated with churn: 21.9% churn with none versus 40.0% with four failures.	Figure 'Churn rate by payment failures'; largest model coefficient (+0.34)
2	Low satisfaction and many support tickets coincide with higher churn: scores 1-2 show 34.6-37.9% churn versus 21.7% at score 4; four or more tickets show 33-37% versus 22-25% for 0-3.	Satisfaction and ticket analyses; coefficients -0.22 and +0.25
3	The model gives only a modest signal: test ROC-AUC 0.558, recall 51.9%, precision 33.5%; actual churn rises from 22.1% (Low band) to 32.1% (High band).	Evaluation table, confusion matrix and risk-band table

Table 16: Findings and evidence

11.2 Recommended retention actions

- **Fix payment friction first:** contact customers with two or more payment failures, offer alternative payment methods and automatic retry or reminders. Expected outcome: lower churn in the group where it is highest.
- **Proactive service recovery:** call customers with four or more support tickets and low satisfaction (1-2) within a set time, and track resolution time. Expected outcome: improved satisfaction and fewer exits.
- **Targeted outreach by risk band:** prioritise the High band for personal contact and the Medium band for low-cost digital offers, and run a controlled pilot (a contacted group versus a comparison group) to measure the real effect. Expected outcome: measured retention uplift and evidence for a cost-benefit decision.

11.3 Future technical enhancements

- **Real-time data pipeline and monitoring:** stream transaction and support events, refresh scores automatically and monitor drift and recall.
- **Richer explainability and modelling:** add SHAP-based per-customer explanations, tune thresholds with real campaign-cost data, and test time-based validation when dated data is available.

11.4 Conclusion

The project delivers a reproducible prototype that goes from raw data to KPIs, model, prediction, recommendation, tests and a container design. It also shows honestly that the supplied data support only a modest churn signal. The behavioural patterns - payment failures, support tickets and low satisfaction - are the most useful business insight, and the prototype is a sound base for a real deployment once richer, time-stamped data and campaign-cost information are available.

Annexure 1: Background of the Organisation

Not applicable. The client, CustomerTech India, is an anonymised organisation provided by the university for an academic case study.

Annexure 2: Data Dictionary

Data name	Aliases	Length / size	Type	Definition	Observed values
Customer_ID	-	6 characters	Alpha-numeric	Unique anonymised customer identifier	C00001 to C02200
Age_Group	Age band	5 categories	Alpha	Customer age band	18-24, 25-34, 35-44, 45-54, 55+
Region	-	5 categories	Alpha	Operating region	Central, East, North, South, West
Customer_Segment	Segment	3 categories	Alpha	Commercial customer segment	Growth, Premium, Value
Tenure_Months	Tenure	Integer	Numeric	Months since acquisition	1 to 60
Monthly_Spend_INR	Spend	Integer	Numeric	Average monthly spend (INR)	605 to 8,996
Transactions_90D	-	Integer	Numeric	Transactions in last 90 days	0 to 18
Support_Tickets_90D	-	Integer	Numeric	Support tickets in last 90 days	0 to 7
Avg_Response_Hours	-	Decimal	Numeric	Average support response time (hours)	1.0 to 30.0
Satisfaction_Score	CSAT	Integer 1-5	Numeric	Latest satisfaction rating	1 to 5
App_Logins_30D	-	Integer	Numeric	App logins in last 30 days	0 to 35
Days_Since_Last_Purchase	Recency	Integer	Numeric	Days since last purchase	1 to 120
Discount_Usage_Pct	-	Integer	Numeric (%)	Share of purchases using discounts	0 to 80
Payment_Failures_90D	-	Integer	Numeric	Failed payments in last 90 days	0 to 4
Churned	Target	3 characters	Binary (Yes/No)	Target variable: churn	Yes 643; No 1,557

Table 17: Data dictionary

Annexure 3: List of Abbreviations, Figures and Tables

Abbreviations: AUC - Area Under the Curve; BCA - Bachelor of Computer Applications; CSAT - Customer Satisfaction; CV - Cross-Validation; DFD - Data Flow Diagram; ERD - Entity-Relationship Diagram; FN/FP/TN/TP - false negative/false positive/true negative/true positive; FR/NFR - functional/non-functional requirement; INR - Indian Rupee; KPI - Key Performance Indicator; PERT - Program Evaluation and Review Technique; PII - Personally Identifiable Information; ROC - Receiver Operating Characteristic; UI - User Interface.

List of figures:

Figure 1: Class balance of the target variable

Figure 2: Churn rate by customer segment
Figure 3: Churn rate by region
Figure 4: Churn rate by payment failures in the previous 90 days
Figure 5: Satisfaction score distribution split by churn status
Figure 6: Churn rate by satisfaction score
Figure 7: Correlation of numeric fields with churn
Figure 8: Confusion matrix on the test set: TN=178, FP=133, FN=62, TP=67
Figure 9: ROC curve on the test set
Figure 10: Logistic Regression coefficients on standardised features
Figure 11: System architecture
Figure 12: Context-level (Level 0) data flow diagram
Figure 13: Level 1 data flow diagram
Figure 14: Entity-relationship diagram
Figure 15: PERT chart (critical path A-B-C-D-E-G-H, 35 days of path duration)
Figure 16: Screen designs: KPI dashboard and churn risk prediction

List of tables:

Table 1: KPIs used in the project
Table 2: Requirements specification with acceptance criteria
Table 3: Profile of the dataset (observed values)
Table 4: Feature preparation decisions
Table 5: Churn rate by customer segment
Table 6: Churn rate by region
Table 7: Churn rate by age group
Table 8: Cross-validation results on the training set (mean of 5 folds)
Table 9: Model evaluation on the held-out test set
Table 10: Actual churn rate by predicted risk band (test set)
Table 11: Activity list for PERT chart
Table 12: Process logic per module
Table 13: Hardware and software
Table 14: Example predictions
Table 15: Test report
Table 16: Findings and evidence
Table 17: Data dictionary

Annexure 4: References

[1] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, 2011, pp. 2825-2830.

[2] W. McKinney, "Data Structures for Statistical Computing in Python," Proceedings of the 9th Python in Science Conference, 2010, pp. 56-61.

[3] D. W. Hosmer, S. Lemeshow and R. X. Sturdivant, Applied Logistic Regression, 3rd ed., Wiley, 2013.

[4] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, 2001, pp. 5-32.

[5] Streamlit documentation, https://docs.streamlit.io

[6] scikit-learn user guide, https://scikit-learn.org/stable/user_guide.html

[7] Docker documentation, https://docs.docker.com

[8] Chandigarh University, Project Submission Guidelines and BCA-S6-MAJOR-AI-001 Student Project Workbook and Development Guide (supplied course documents).

[9] Client Data Set, Data Dictionary and templates supplied with the project (anonymised, academic use only).

Annexure 5: Soft Copy of the Project

The complete source code, data, model artefact, documentation and this report are provided in the repository archive churn-retention-platform.zip. Upload it using the drive link shared by the university.

Annexure 6: Guide Details

Guide name	_____
Full address	_____
Qualification	_____
Mobile	_____
Email	_____

Annexure 7: Certificate from Guide

See the Certificate from Guide at the front of this report (Annexure A format).

Annexure 8: Source Code Printout

src/data_prep.py

```
"""Data preparation: loading, cleaning, validation and feature pipeline.
This module never imports app.py (module boundary: data layer -> no UI dependency)."""
from pathlib import Path
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.model_selection import train_test_split

ROOT = Path(__file__).resolve().parents[1]
RAW_PATH = ROOT / "data" / "raw" / "customer_data.xlsx"
CLEAN_PATH = ROOT / "data" / "processed" / "customer_data_clean.csv"

ID_COL = "Customer_ID"
TARGET = "Churned"
CATEGORICAL = ["Age_Group", "Region", "Customer_Segment"]
NUMERIC = ["Tenure_Months", "Monthly_Spend_INR", "Transactions_90D", "Support_Tickets_90D",
           "Avg_Response_Hours", "Satisfaction_Score", "App_Logins_30D",
           "Days_Since_Last_Purchase", "Discount_Usage_Pct", "Payment_Failures_90D"]
FEATURES = CATEGORICAL + NUMERIC
RANDOM_STATE = 42


def load_raw(path=RAW_PATH) -> pd.DataFrame:
    return pd.read_excel(path)


def validate(df: pd.DataFrame) -> dict:
    """Return a dict of data-quality findings (used by tests and docs)."""
    return {
        "rows": len(df),
        "missing_values": int(df.isna().sum().sum()),
        "duplicate_rows": int(df.duplicated().sum()),
        "duplicate_ids": int(df[ID_COL].duplicated().sum()),
        "discount_out_of_bounds": int(((df["Discount_Usage_Pct"] < 0) | (df["Discount_Usage_Pct"] > 100)).sum()),
        "satisfaction_out_of_bounds": int(((df["Satisfaction_Score"] < 1) | (df["Satisfaction_Score"] > 5)).sum()),
        "negative_numeric": int((df[NUMERIC] < 0).sum().sum()),
        "invalid_target": int((~df[TARGET].isin(["Yes", "No"])).sum()),
    }


def clean(df: pd.DataFrame) -> pd.DataFrame:
    """Light cleaning: strip text, drop duplicates, impute numeric gaps with median
    (none are expected in the supplied data, but the step keeps the pipeline robust)."""
    df = df.copy()
    for c in CATEGORICAL + [ID_COL, TARGET]:
        df[c] = df[c].astype(str).str.strip()
    df = df.drop_duplicates(subset=ID_COL)
    for c in NUMERIC:
        if df[c].isna().any():
            df[c] = df[c].fillna(df[c].median())
    return df.reset_index(drop=True)


def encode_target(series: pd.Series) -> pd.Series:
    return (series == "Yes").astype(int)


def build_preprocessor() -> ColumnTransformer:
    """One-hot encode nominal categories (no natural order in Region/Segment; age bands are
    handled the same way for simplicity) and standardise numeric fields (needed for Logistic
    Regression; harmless for tree models). Customer_ID is excluded by construction."""
    return ColumnTransformer([
        ("cat", OneHotEncoder(handle_unknown="ignore"), CATEGORICAL),
        ("num", StandardScaler(), NUMERIC),
    ])


def get_split(df: pd.DataFrame, test_size=0.2):
    """Stratified split so train and test keep the same churn proportion."""
    X, y = df[FEATURES], encode_target(df[TARGET])
    return train_test_split(X, y, test_size=test_size, stratify=y, random_state=RANDOM_STATE)


def main():
    df = clean(load_raw())
    print("Validation:", validate(df))
    CLEAN_PATH.parent.mkdir(parents=True, exist_ok=True)
    df.to_csv(CLEAN_PATH, index=False)
    print(f"Saved {CLEAN_PATH} ({len(df)} rows)")


if __name__ == "__main__":
    main()
```

src/train_model.py

```
"""Model training: compares candidate classifiers with stratified CV on the TRAINING set only,
then fits the chosen model and serialises it to models/churn_model.pkl."""
import json
import joblib
import numpy as np
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.model_selection import StratifiedKFold, cross_validate
from src.data_prep import (ROOT, load_raw, clean, get_split, build_preprocessor, RANDOM_STATE)

MODEL_PATH = ROOT / "models" / "churn_model.pkl"
CV_PATH = ROOT / "models" / "cv_results.json"

CANDIDATES = {
    "Logistic Regression": LogisticRegression(max_iter=1000, class_weight="balanced", random_state=RANDOM_STATE),
    "Random Forest": RandomForestClassifier(n_estimators=300, max_depth=6, min_samples_leaf=10,
                                            class_weight="balanced", random_state=RANDOM_STATE, n_jobs=-1),
    "Gradient Boosting": GradientBoostingClassifier(n_estimators=150, max_depth=2, learning_rate=0.05,
                                                    random_state=RANDOM_STATE),
}
CHOSEN = "Logistic Regression"


def make_pipeline(name):
    return Pipeline([("prep", build_preprocessor()), ("clf", CANDIDATES[name])])


def main():
    df = clean(load_raw())
    X_train, X_test, y_train, y_test = get_split(df)
    print(f"Train {len(X_train)} rows, test {len(X_test)} rows, train churn rate {y_train.mean():.3f}, test {y_test.mean():.3f}")
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
    results = {}
    for name in CANDIDATES:
        s = cross_validate(make_pipeline(name), X_train, y_train, cv=cv,
                           scoring=["roc_auc", "recall", "precision", "f1", "accuracy"])
        results[name] = {k.replace("test_", ""): float(np.mean(v)) for k, v in s.items() if k.startswith("test_")}
        print(name, {k: round(v, 3) for k, v in results[name].items()})
    CV_PATH.write_text(json.dumps(results, indent=2))
    model = make_pipeline(CHOSEN).fit(X_train, y_train)
    joblib.dump(model, MODEL_PATH)
    print(f"Saved {CHOSEN} to {MODEL_PATH}")


if __name__ == "__main__":
    main()
```

src/evaluate_model.py

```
"""Evaluate the saved model on the held-out test set; write metrics + figures."""
import json
import joblib
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             confusion_matrix, roc_auc_score, roc_curve)
from src.data_prep import ROOT, load_raw, clean, get_split
from src.train_model import MODEL_PATH

DOCS = ROOT / "docs"
FIG = DOCS / "figures"


def main():
    FIG.mkdir(parents=True, exist_ok=True)
    df = clean(load_raw())
    X_train, X_test, y_train, y_test = get_split(df)
    model = joblib.load(MODEL_PATH)
    proba = model.predict_proba(X_test)[:, 1]
    pred = (proba >= 0.5).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_test, pred).ravel()
    m = {
        "n_test": int(len(y_test)), "accuracy": accuracy_score(y_test, pred),
        "precision": precision_score(y_test, pred), "recall": recall_score(y_test, pred),
        "f1": f1_score(y_test, pred), "roc_auc": roc_auc_score(y_test, proba),
        "tn": int(tn), "fp": int(fp), "fn": int(fn), "tp": int(tp),
        "majority_baseline_accuracy": float(1 - y_test.mean()),
    }
    # Risk-band distribution on test set (Low <0.35, Medium 0.35-0.55, High >=0.55)
    bands = np.where(proba >= 0.55, "High", np.where(proba >= 0.35, "Medium", "Low"))
    m["bands"] = {b: {"n": int((bands == b).sum()), "actual_churn_rate": float(y_test[bands == b].mean())}
                  for b in ["Low", "Medium", "High"] if (bands == b).any()}
```

```
    # Coefficients
    prep, clf = model.named_steps["prep"], model.named_steps["clf"]
    names = [n.split("__")[1] for n in prep.get_feature_names_out()]
    coefs = sorted(zip(names, clf.coef_[0]), key=lambda t: t[1])
    m["coefficients"] = {n: float(c) for n, c in coefs}
    (ROOT / "models" / "metrics.json").write_text(json.dumps(m, indent=2))
    print(json.dumps({k: v for k, v in m.items() if k != "coefficients"}, indent=2))

    fig, ax = plt.subplots(figsize=(4.2, 3.6))
    cm = np.array([[tn, fp], [fn, tp]])
    ax.imshow(cm, cmap="Blues")
    for i in range(2):
        for j in range(2):
            ax.text(j, i, cm[i, j], ha="center", va="center", fontsize=16,
                    color="white" if cm[i, j] > cm.max() / 2 else "black")
    ax.set_xticks([0, 1]); ax.set_yticks([0, 1])
    ax.set_xticklabels(["Pred: Stay", "Pred: Churn"]); ax.set_yticklabels(["Actual: Stay", "Actual: Churn"])
    ax.set_title("Confusion matrix (test set, n=%d)" % len(y_test), fontsize=10)
    fig.tight_layout(); fig.savefig(FIG / "confusion_matrix.png", dpi=180); plt.close(fig)

    fpr, tpr, _ = roc_curve(y_test, proba)
    fig, ax = plt.subplots(figsize=(4.2, 3.6))
    ax.plot(fpr, tpr, color="#1f4e79", label=f"AUC = {m['roc_auc']:.3f}")
    ax.plot([0, 1], [0, 1], "--", color="grey", label="Random guess")
    ax.set_xlabel("False positive rate"); ax.set_ylabel("True positive rate")
    ax.set_title("ROC curve (test set)", fontsize=10); ax.legend()
    fig.tight_layout(); fig.savefig(FIG / "roc_curve.png", dpi=180); plt.close(fig)

    fig, ax = plt.subplots(figsize=(6, 4.4))
    ns, cs = zip(*coefs)
    ax.barh(ns, cs, color=["#c0392b" if c > 0 else "#2e86c1" for c in cs])
    ax.axvline(0, color="black", lw=0.8)
    ax.set_title("Logistic Regression coefficients (standardised features)", fontsize=10)
    ax.set_xlabel("Coefficient (red = raises churn risk)")
    fig.tight_layout(); fig.savefig(FIG / "coefficients.png", dpi=180); plt.close(fig)


if __name__ == "__main__":
    main()
```

src/predict.py

```
"""Business-logic / model-serving layer: prediction, risk bands, recommended actions."""
import os
import joblib
import pandas as pd
from src.data_prep import FEATURES, CATEGORICAL, NUMERIC, ROOT

MODEL_PATH = os.environ.get("MODEL_PATH", str(ROOT / "models" / "churn_model.pkl"))
LOW_MAX, HIGH_MIN = 0.35, 0.55

ACTIONS = {
    "Low": "Maintain normal engagement: include in regular newsletters and loyalty communication.",
    "Medium": "Proactive check-in: send a personalised offer or app-engagement nudge and review recent support tickets.",
    "High": "Priority retention call: assign an account owner, resolve open support/payment issues and offer a tailored incentive.",
}
BOUNDS = {"Tenure_Months": (0, 120), "Monthly_Spend_INR": (0, 1_000_000), "Transactions_90D": (0, 1000),
          "Support_Tickets_90D": (0, 1000), "Avg_Response_Hours": (0, 1000), "Satisfaction_Score": (1, 5),
          "App_Logins_30D": (0, 1000), "Days_Since_Last_Purchase": (0, 10000),
          "Discount_Usage_Pct": (0, 100), "Payment_Failures_90D": (0, 1000)}

_model = None


def load_model():
    global _model
    if _model is None:
        _model = joblib.load(MODEL_PATH)
    return _model


def risk_band(p: float) -> str:
    return "High" if p >= HIGH_MIN else "Medium" if p >= LOW_MAX else "Low"


def validate_input(record: dict):
    missing = [f for f in FEATURES if f not in record or record[f] is None]
    if missing:
        raise ValueError(f"Missing fields: {missing}")
    for c, (lo, hi) in BOUNDS.items():
        if not lo <= record[c] <= hi:
            raise ValueError(f"{c}={record[c]} outside allowed range {lo}-{hi}")


def predict_one(record: dict) -> dict:
    validate_input(record)
    p = float(load_model().predict_proba(pd.DataFrame([record])[FEATURES])[:, 1][0])
    band = risk_band(p)
    return {"probability": p, "band": band, "action": ACTIONS[band]}
```

```
def top_drivers(record: dict, n=3):
    """Return the n features pushing this customer's score up the most (coef x standardised value)."""
    m = load_model(); prep, clf = m.named_steps["prep"], m.named_steps["clf"]
    x = prep.transform(pd.DataFrame([record])[FEATURES])
    x = x.toarray() if hasattr(x, "toarray") else x
    names = [s.split("__")[1] for s in prep.get_feature_names_out()]
    contrib = sorted(zip(names, (x[0] * clf.coef_[0])), key=lambda t: -t[1])
    return [(a, float(b)) for a, b in contrib[:n]]
```

src/app.py

```
"""Streamlit UI layer. Run: streamlit run src/app.py"""
import sys
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
import pandas as pd
import streamlit as st
import plotly.express as px
from src.data_prep import CLEAN_PATH, load_raw, clean, CATEGORICAL
from src.predict import predict_one, top_drivers

st.set_page_config(page_title="Customer Retention Platform", layout="wide")
st.title("AI-Powered Customer Intelligence and Retention Platform")
st.caption("Academic prototype - CustomerTech India (anonymised). Predictions are decision support, not guaranteed outcomes.")


@st.cache_data
def get_data():
    df = pd.read_csv(CLEAN_PATH) if Path(CLEAN_PATH).exists() else clean(load_raw())
    df["Churn_Flag"] = (df["Churned"] == "Yes").astype(int)
    return df


df = get_data()
tab1, tab2 = st.tabs(["KPI Dashboard", "Churn Risk Prediction"])

with tab1:  # FR-01, FR-04
    st.sidebar.header("Dashboard filters")
    seg = st.sidebar.multiselect("Segment", sorted(df.Customer_Segment.unique()), default=sorted(df.Customer_Segment.unique()))
    reg = st.sidebar.multiselect("Region", sorted(df.Region.unique()), default=sorted(df.Region.unique()))
    f = df[df.Customer_Segment.isin(seg) & df.Region.isin(reg)]
    if f.empty:
        st.warning("No customers match the selected filters.")
    else:
        c1, c2, c3, c4 = st.columns(4)
        c1.metric("Customers", f"{len(f):,}")
        c2.metric("Churn rate", f"{f.Churn_Flag.mean()*100:.1f}%")
        c3.metric("Avg satisfaction (churned)", f"{f[f.Churn_Flag==1].Satisfaction_Score.mean():.2f}" if (f.Churn_Flag==1).any() els
        c4.metric("Avg monthly spend (INR)", f"{f.Monthly_Spend_INR.mean():,.0f}")
        a, b = st.columns(2)
        for col, container in [("Customer_Segment", a), ("Region", b)]:
            g = f.groupby(col).Churn_Flag.mean().mul(100).reset_index()
            container.plotly_chart(px.bar(g, x=col, y="Churn_Flag", labels={"Churn_Flag": "Churn rate (%)"},
                                          title=f"Churn rate by {col}"), width="stretch")
        g = f.groupby("Payment_Failures_90D").Churn_Flag.mean().mul(100).reset_index()
        st.plotly_chart(px.bar(g, x="Payment_Failures_90D", y="Churn_Flag", labels={"Churn_Flag": "Churn rate (%)"},
                               title="Churn rate by payment failures (90 days)"), width="stretch")

with tab2:  # FR-02, FR-03, FR-05, FR-06
    st.subheader("Enter customer attributes (no personal identifiers are collected)")
    with st.form("customer"):
        c1, c2, c3 = st.columns(3)
        age = c1.selectbox("Age group", ["18-24", "25-34", "35-44", "45-54", "55+"])
        region = c2.selectbox("Region", ["East", "West", "North", "Central", "South"])
        segment = c3.selectbox("Segment", ["Value", "Growth", "Premium"])
        tenure = c1.number_input("Tenure (months)", 0, 120, 24)
        spend = c2.number_input("Monthly spend (INR)", 0, 1_000_000, 4800)
        txn = c3.number_input("Transactions (90d)", 0, 1000, 9)
        tickets = c1.number_input("Support tickets (90d)", 0, 1000, 3)
        resp = c2.number_input("Avg response hours", 0.0, 1000.0, 15.0)
        sat = c3.slider("Satisfaction score", 1, 5, 3)
        logins = c1.number_input("App logins (30d)", 0, 1000, 17)
        recency = c2.number_input("Days since last purchase", 0, 10000, 60)
        disc = c3.number_input("Discount usage (%)", 0, 100, 40)
        fails = c1.number_input("Payment failures (90d)", 0, 1000, 2)
        go = st.form_submit_button("Predict churn risk")
    if go:
        rec = {"Age_Group": age, "Region": region, "Customer_Segment": segment, "Tenure_Months": tenure,
               "Monthly_Spend_INR": spend, "Transactions_90D": txn, "Support_Tickets_90D": tickets,
               "Avg_Response_Hours": resp, "Satisfaction_Score": sat, "App_Logins_30D": logins,
               "Days_Since_Last_Purchase": recency, "Discount_Usage_Pct": disc, "Payment_Failures_90D": fails}
        try:
            out = predict_one(rec)
            st.metric("Churn probability", f"{out['probability']*100:.1f}%")
            {"Low": st.success, "Medium": st.warning, "High": st.error}[out["band"]](f"Risk band: {out['band']}")
            st.info("Recommended action: " + out["action"])
            st.write("Top factors raising this customer's score:",
                     ", ".join(f"{n} ({v:+.2f})" for n, v in top_drivers(rec)))
            st.caption("Model is a weak-to-moderate signal (see report); use alongside human judgement.")
```

```
        except ValueError as e:
            st.error(str(e))
```

tests/test_platform.py

```
import pandas as pd, pytest, joblib
from src import data_prep as dp
from src.predict import predict_one, risk_band, validate_input
from src.train_model import MODEL_PATH, make_pipeline

BASE = {"Age_Group": "25-34", "Region": "East", "Customer_Segment": "Value", "Tenure_Months": 24,
        "Monthly_Spend_INR": 4800, "Transactions_90D": 9, "Support_Tickets_90D": 3, "Avg_Response_Hours": 15.0,
        "Satisfaction_Score": 3, "App_Logins_30D": 17, "Days_Since_Last_Purchase": 60,
        "Discount_Usage_Pct": 40, "Payment_Failures_90D": 2}

def test_tc01_data_loads():
    df = dp.load_raw(); assert df.shape == (2200, 15) and list(df.columns[:1]) == ["Customer_ID"]

def test_tc02_missing_values_handled():
    df = dp.load_raw(); df.loc[0, "Monthly_Spend_INR"] = None
    out = dp.clean(df); assert out["Monthly_Spend_INR"].isna().sum() == 0

def test_tc03_encoding_excludes_id_and_shape():
    df = dp.clean(dp.load_raw()); pre = dp.build_preprocessor().fit(df[dp.FEATURES])
    names = pre.get_feature_names_out()
    assert not any("Customer_ID" in n for n in names) and len(names) == 5 + 5 + 3 + 10

def test_tc04_stratified_split():
    df = dp.clean(dp.load_raw()); _, _, ytr, yte = dp.get_split(df)
    assert abs(ytr.mean() - yte.mean()) < 0.01 and len(yte) == 440

def test_tc05_training_runs():
    df = dp.clean(dp.load_raw()); Xtr, _, ytr, _ = dp.get_split(df)
    m = make_pipeline("Logistic Regression").fit(Xtr, ytr); assert hasattr(m, "predict_proba")

def test_tc06_prediction_valid_probability():
    out = predict_one(BASE); assert 0 <= out["probability"] <= 1 and out["band"] in {"Low", "Medium", "High"}

def test_tc07_risk_bands_and_actions():
    assert (risk_band(0.1), risk_band(0.4), risk_band(0.9)) == ("Low", "Medium", "High")
    assert predict_one(BASE)["action"]

def test_tc08_dashboard_filter_logic():
    df = dp.clean(dp.load_raw()); f = df[(df.Customer_Segment == "Premium") & (df.Region == "West")]
    assert 0 < len(f) < len(df) and set(f.Customer_Segment) == {"Premium"}

def test_tc09_edge_zero_tenure():
    r = dict(BASE, Tenure_Months=0); assert 0 <= predict_one(r)["probability"] <= 1

def test_tc10_invalid_input_rejected():
    with pytest.raises(ValueError): validate_input(dict(BASE, Satisfaction_Score=9))
    with pytest.raises(ValueError): validate_input({k: v for k, v in BASE.items() if k != "Region"})

def test_tc11_no_pii_in_model_inputs():
    assert "Customer_ID" not in dp.FEATURES

def test_tc12_data_quality_clean():
    v = dp.validate(dp.load_raw()); assert v["missing_values"] == 0 and v["duplicate_ids"] == 0 and v["discount_out_of_bounds"] == 0
```

Dockerfile

```
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 MODEL_PATH=/app/models/churn_model.pkl
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir pandas numpy scikit-learn streamlit plotly openpyxl joblib matplotlib
COPY src/ src/
COPY models/churn_model.pkl models/churn_model.pkl
COPY data/processed/customer_data_clean.csv data/processed/customer_data_clean.csv
RUN useradd -m appuser && chown -R appuser /app
USER appuser
EXPOSE 8501
HEALTHCHECK CMD python -c "import urllib.request;urllib.request.urlopen('http://localhost:8501/_stcore/health')"
CMD ["streamlit", "run", "src/app.py", "--server.port=8501", "--server.address=0.0.0.0"]
```